Міністерство освіти і науки України
Національний університет «Полтавська політехніка імені Юрія Кондратюка»
Навчально-науковий інститут інформаційних технологій та робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем

ЗВІТ

з лабораторної роботи №2
з дисципліни «Технології на платформі .NET»

на тему: «Асинхронний потокобезпечний CRUD-сервіс, паралельна обробка даних та примітиви синхронізації»

Виконав: студент групи 402-ТН
Желудков Віталій Віталійович

Перевірив: Тютюнник П. Б.

Полтава — 2026

1. Опис виконаної роботи

Роботу виконано в гілці `lab2`, створеній від гілки `lab1` (вона містить код першої лабораторної роботи). Тема моделі та сама — «Ігрова студія» (GameStudio), .NET 8, C#. Рішення `GameStudio.sln` тепер містить три проєкти: `GameStudio.Common`, `GameStudio.Console` і новий проєкт модульних тестів `GameStudio.Tests`.

Нові та змінені файли:

Файл	Що зроблено
`Interfaces/ICrudServiceAsync.cs`	інтерфейс асинхронного CRUD-сервісу з завдання (наслідує `IEnumerable<T>`)
`Services/CrudServiceAsync.cs`	асинхронна потокобезпечна реалізація: `ConcurrentDictionary`, пагінація, `SaveAsync`/`LoadAsync`
`Generation/RandomData.cs`	генератор випадкових даних (імена, назви ігор, суми, дати) на основі потокобезпечного `Random.Shared`
`Models/Programmer.cs`, `Artist.cs`, `GameProject.cs`, `BugReport.cs`	статичні методи `CreateNew()`, які створюють об'єкти зі згенерованими даними
`Models/Employee.cs`	статичний лічильник об'єктів тепер збільшується через `Interlocked.Increment` (безпечно при паралельному створенні)
`Console/Demos/Lab2Demo.cs`	паралельне створення понад 1000 об'єктів, статистика LINQ, пагінація, CRUD, збереження у файли
`Console/Demos/SyncPrimitivesDemo.cs`	приклади примітивів синхронізації
`Console/Demos/Lab1Demo.cs`, `ConsoleUi.cs`, `Program.cs`	демонстрацію ЛР1 перенесено в окремий клас (запуск з аргументом `lab1`), спільні методи виводу
`GameStudio.Tests/*`	модульні тести NUnit (додаткове завдання)

1.1. Асинхронний потокобезпечний CRUD-сервіс CrudServiceAsync<T>

- **Сховище** — потокобезпечна вбудована колекція `ConcurrentDictionary<Guid, StoredItem>`; операції `TryAdd`, `TryUpdate`, `TryRemove` атомарні, тому окреме блокування для CRUD-операцій не потрібне.
- **Порядок елементів** — разом з елементом зберігається порядковий номер, який видається атомарно через `Interlocked.Increment`. Завдяки цьому `ReadAllAsync`, перебір і пагінація завжди повертають елементи в порядку додавання.
- **Методи CRUD** повертають `Task<bool>`: `true` — операцію виконано, `false` — елемент уже існує (Create) або не знайдений (Update, Remove). `ReadAsync` для відсутнього Id повертає задачу з `KeyNotFoundException`. `UpdateAsync` використовує оптимістичне блокування: `TryUpdate` у циклі до успіху.
- **Пагінація** — `ReadAllAsync(page, amount)`: сторінки нумеруються з 1, повертається `Skip((page - 1) * amount).Take(amount)`; некоректні аргументи дають `ArgumentOutOfRangeException`, сторінка за межами — порожню колекцію; `GetTotalPages(amount)` рахує кількість сторінок.
- **Збереження** — `SaveAsync()` асинхронно серіалізує знімок колекції в JSON (`JsonSerializer.SerializeAsync`, `FileStream` з `useAsync: true`) у файл `FilePath`. Доступ до файлу захищено `SemaphoreSlim` (окремий семафор на кожен шлях, спільний для всіх екземплярів сервісу), запис іде в тимчасовий файл, який потім

атомарно підміняє основний (`File.Move(..., overwrite: true)`). Додатковий метод `LoadAsync()` завантажує колекцію з файлу.

- **IEnumerable<T>** — `GetEnumerator()` перебирає знімок (копію) колекції, тому одночасні зміни з інших потоків не викликають `InvalidOperationException`.

1.2. Паралельна генерація даних і статистика

Статичні методи `Programmer.CreateNew()`, `Artist.CreateNew()`, `GameProject.CreateNew()` і `BugReport.CreateNew(projectId)` створюють об'єкти з випадковими даними. Консольний застосунок паралельно створює 1000 працівників (`Parallel.For`), 1000 проєктів (`Parallel.ForEachAsync`) і 1–2 баги для кожного проєкту (`Parallel.ForEach`), додаючи їх через `CrudServiceAsync`. Для числових властивостей за допомогою LINQ (`Min`, `Max`, `Average`) обчислюються мінімальні, максимальні та середні значення, згенеровані колекції зберігаються у файли `data/lab2/*.json`.

1.3. Примітиви синхронізації

Примітив	Сценарій у програмі
`lock`, `Interlocked`	лічильник очок: 1 000 000 інкрементів без синхронізації (стан гонки), з `lock` і з `Interlocked`
`Monitor.TryEnter`	спроба зайняти слот збереження гри з тайм-аутом
`Semaphore`	рендер-ферма на 2 машини: 6 потоків, одночасно працюють не більше 2
`SemaphoreSlim`	8 асинхронних завантажень (`WaitAsync`), не більше 3 одночасно; також захищає файл у `CrudServiceAsync`
`AutoResetEvent`	сервер збірок отримує запити по одному; один `Set()` пропускає лише один потік
`ManualResetEventSlim`, `CountdownEvent`	старт матчу: один `Set()` відпускає всіх 4 гравців; очікування, поки стартують усі
`ReaderWriterLockSlim`	таблиця рекордів: паралельні читання, ексклюзивний запис
`Barrier`	фізика, ШІ та рендер синхронізуються в кінці кожного кадру
`Mutex`	іменований м'ютекс для перевірки, що запущено один екземпляр програми

1.4. Модульні тести (додаткове завдання)

Проєкт `GameStudio.Tests` (NUnit) містить 39 тестів: створення, читання, оновлення й видалення, пагінація (параметризовані тести `[TestCase]`), перебір сервісу, паралельні операції з багатьох потоків, збереження та завантаження з файлу (зокрема 20 одночасних `SaveAsync`), а також перевірка методів `CreateNew` і потокобезпечності статичного лічильника. Тести запускаються у Visual Studio (Test Explorer) або командою `dotnet test`.

Гілка lab2:
https://github.com/vitalikcraft666-design/NUPP_NET_2026_402_TK_Zheludkov_Lab/tree/lab2

Репозиторій:
https://github.com/vitalikcraft666-design/NUPP_NET_2026_402_TK_Zheludkov_Lab

2. Результати виконання консольного застосунку

Команда запуску: `dotnet run --project GameStudio.Console`

```
Лабораторна робота №2 — асинхронний потокобезпечний CRUD-сервіс (GameStudio)
Запуск: 22.10.2026 00:11; логічних процесорів: 2

=============================================================================================
1. Паралельне створення об'єктів (клас Parallel + статичні методи CreateNew + CrudServiceAsync)
=============================================================================================
Працівників (Parallel.For):         додано 1000 з 1000
Проєктів (Parallel.ForEachAsync):   додано 1000 з 1000
Багів (Parallel.ForEach, 1–2 на проєкт): додано 1503 з 1503
Усього об'єктів: 3503; задіяно потоків: 3; час: 46 мс
Статичний лічильник Employee: +1000 (збігається з кількістю створених працівників завдяки Interlocked.Increment)

=============================================================================================
2. Статистика згенерованих даних: мінімальні, максимальні та середні значення (LINQ)
=============================================================================================
Працівники: 1000 (програмістів: 491, художників: 509)
  Показник                                       Мін           Макс        Середнє
  ---------------------------------------------------------------------------------
  Базова ставка, грн                      18 000,00      60 000,00      37 131,00
  Місячна зарплата, грн                   19 330,00     186 875,00      63 635,92
  Стаж, років                                   0,0           11,0            5,7
  Коміти (програмісти)                         21,0        4 998,0        2 432,4
  Завершені роботи (художники)                  0,0           40,0           19,8

Проєкти: 1000
  Показник                                       Мін           Макс        Середнє
  ---------------------------------------------------------------------------------
  Бюджет, грн                             51 000,00   1 999 000,00   1 026 060,00
  Витрачено, грн                             216,58   2 266 362,92     628 766,21
  Використання бюджету, %                       0,1          119,9           60,6
  Розмір команди, осіб                          2,0           15,0            8,6

Баги: 1503
  Показник                                       Мін           Макс        Середнє
  ---------------------------------------------------------------------------------
  Вік бага, днів                                0,0           90,0           45,4
  Багів на один проєкт                         1,00           2,00           1,50

Програмісти за грейдом: Junior — 116, Middle — 130, Senior — 126, Lead — 119
Проєкти за статусом:    Concept — 160, Prototype — 158, Production — 180, Testing — 171, Released — 176, Cancelled — 155
Баги за критичністю:    Low — 386, Medium — 395, High — 377, Critical — 345; виправлено: 24,4 %
Найпопулярніші мови:    JavaScript — 68, GDScript — 68, Python — 65

Топ-3 проєкти за бюджетом (OrderByDescending + Take):
  1. "Star Tactics" (Puzzle, Unreal Engine) — Prototype, бюджет: 1 999 000 грн, витрачено: 786 148 грн, у команді: 6
  2. "Lost Odyssey" (Horror, Godot) — Cancelled, бюджет: 1 997 000 грн, витрачено: 1 811 673 грн, у команді: 9
  3. "Star Pirates" (Horror, Godot) — Released, бюджет: 1 996 000 грн, витрачено: 1 323 045 грн, у команді: 12

=============================================================================================
3. Пагінація: ReadAllAsync(page, amount), розмір сторінки — 5
=============================================================================================
Працівників: 1000, сторінок по 5: 200
Сторінка 1:
  1. Марія Руденко (UI-художник), 56 500,00 грн
  2. Назар Марченко (3D-моделер), 59 710,00 грн
  3. Анна Гнатюк (Senior Python-програміст), 105 410,00 грн
```

```
  4. Роман Кравченко (Концепт-художник), 66 510,00 грн
  5. Дарина Романюк (3D-моделер), 33 120,00 грн
Сторінка 2:
  1. Марія Коваленко (Middle JavaScript-програміст), 69 000,00 грн
  2. Катерина Кравченко (Концепт-художник), 53 770,00 грн
  3. Роман Литвин (UI-художник), 33 750,00 грн
  4. Юлія Коваленко (Senior Rust-програміст), 110 500,00 грн
  5. Назар Мороз (Аніматор), 52 480,00 грн
Сторінка 200:
  1. Юлія Олійник (Lead Luau-програміст), 162 137,50 грн
  2. Андрій Мороз (Аніматор), 35 430,00 грн
  3. Олександр Руденко (Концепт-художник), 31 050,00 грн
  4. Назар Павленко (Middle GDScript-програміст), 74 100,00 грн
  5. Богдан Мороз (Senior C#-програміст), 124 630,00 грн
Сторінка 201 (за межами): елементів — 0
ReadAllAsync(0, 5) → ArgumentOutOfRangeException: Номер сторінки має бути не менше 1. (Parameter 'page')

================================================================================================
4. Асинхронні операції CRUD та перебір сервісу як IEnumerable<T>
================================================================================================
ReadAsync(Id)               → Анна Гнатюк (Senior Python-програміст)
UpdateAsync (підвищення)    → True; новий грейд: Lead Python-програміст
UpdateAsync (нема в сервісі)→ False
CreateAsync (дублікат Id)   → False
RemoveAsync                 → True; повторно → False
ReadAsync(Guid.Empty)       → KeyNotFoundException: Елемент з Id 00000000-0000-0000-0000-000000000000 не знайдено.
LINQ по сервісу: програмістів — 490, художників — 509, усього — 999
foreach по сервісу (перші 3):
  - Марія Руденко (UI-художник)
  - Назар Марченко (3D-моделер)
  - Анна Гнатюк (Lead Python-програміст)

================================================================================================
5. Потокобезпечність: одночасні Create, Remove та перебір з різних потоків (Task.Run + Task.WhenAll)
================================================================================================
Було: 999; створено: 300; видалено: 100; повних переборів під час змін: 50
Стало: 1199 (очікувалось 1199) — дані узгоджені, винятків немає

================================================================================================
6. Асинхронне збереження згенерованих колекцій у файли (SaveAsync) і завантаження (LoadAsync)
================================================================================================
SaveAsync трьох сервісів одночасно → True, True, True
  data/lab2/employees.json — 335 496 байт
  data/lab2/projects.json — 636 383 байт
  data/lab2/bugs.json — 496 575 байт
10 одночасних SaveAsync в один файл → успішних: 10 з 10
LoadAsync у нові сервіси → True, True, True; працівників: 1199, проєктів: 1000, багів: 1503
Порядок елементів збережено: True; типи Programmer/Artist збережено: True; фонд оплати праці збігається: True

================================================================================================
7. Примітиви синхронізації
================================================================================================

--- 7.1 Стан гонки vs lock vs Interlocked (лічильник очок, 1 000 000 інкрементів з різних потоків)
    без синхронізації: 596 078 (втрачено оновлень: 403 922)
    lock:              1 000 000
    Interlocked:       1 000 000

--- 7.2 Monitor.TryEnter — спроба зайняти слот збереження гри з тайм-аутом
    TryEnter(50 мс) під час автозбереження → False (слот зайнятий, потік не завис)
```

```
    TryEnter(2 с)                        → True (дочекались, поки автозбереження звільнить слот)

--- 7.3 Semaphore — рендер-ферма на 2 машини, 6 сцен у 6 окремих потоках (Thread)
    сцена 3: рендер почато у потоці 10, зайнято машин: 1
    сцена 1: рендер почато у потоці 8, зайнято машин: 2
    сцена 4: рендер почато у потоці 11, зайнято машин: 1
    сцена 5: рендер почато у потоці 12, зайнято машин: 2
    сцена 2: рендер почато у потоці 9, зайнято машин: 1
    сцена 6: рендер почато у потоці 13, зайнято машин: 2
    максимум одночасно працювало: 2 з 6 потоків

--- 7.4 SemaphoreSlim + WaitAsync — 8 асинхронних завантажень ресурсів, не більше 3 одночасно
    завантажено 8 ресурсів; максимум одночасних завантажень: 3

--- 7.5 AutoResetEvent — сервер збірок обробляє запити по одному
    головний потік: Set() — запит на збірку Windows
    сервер збірок: отримав сигнал → збираю Windows
    головний потік: Set() — запит на збірку Android
    сервер збірок: отримав сигнал → збираю Android
    головний потік: Set() — запит на збірку WebGL
    сервер збірок: отримав сигнал → збираю WebGL
    3 потоки чекають; після одного Set() пройшов 1; решта — після ще 2 Set() (усього пройшло 3)

--- 7.6 ManualResetEventSlim + CountdownEvent — старт матчу для 4 гравців
    до Set(): стартувало гравців — 0
    після одного Set(): стартували всі 4; IsSet = True; CountdownEvent.CurrentCount = 0

--- 7.7 ReaderWriterLockSlim — таблиця рекордів: 36 читань і 4 записи паралельно
    читань: 36, записів: 4; одночасних читачів (макс.): 2; одночасних письменників (макс.): 1; рекорд: 8754

--- 7.8 Barrier — фізика, ШІ та рендер синхронізуються в кінці кожного кадру
    кадр 1: усі 3 підсистеми завершили роботу → наступний кадр
    кадр 2: усі 3 підсистеми завершили роботу → наступний кадр
    кадр 3: усі 3 підсистеми завершили роботу → наступний кадр

--- 7.9 Mutex — перевірка, що запущено лише один екземпляр програми
    м'ютекс створено цим процесом: True; захоплено: True → запущено єдиний екземпляр

================================================================================================
Підсумок
================================================================================================
Працівників у сервісі: 1199, проєктів: 1000, багів: 1503
Усього створено об'єктів Employee за час роботи (разом із завантаженими з файлу): 2500
Роботу програми завершено.
```

3. Результати модульних тестів

Усі 39 тестів пройшли успішно.

```
✔ CreateNewTests.Artist_CreateNew_GeneratesValidData
✔ CreateNewTests.BugReport_CreateNew_WithProjectId_LinksBugToProject
✔ CreateNewTests.CreateNew_CalledInParallel_ProducesUniqueObjects
✔ CreateNewTests.Employee_CreatedCounter_IsThreadSafe
✔ CreateNewTests.GameProject_CreateNew_GeneratesValidData
✔ CreateNewTests.Programmer_CreateNew_GeneratesValidData
✔ CrudServiceAsyncTests.ConcurrentCreateAndRemove_KeepsCountConsistent
✔ CrudServiceAsyncTests.Constructor_EmptyFilePath_ThrowsArgumentException
✔ CrudServiceAsyncTests.CreateAsync_DuplicateId_ReturnsFalseAndKeepsOriginal
✔ CrudServiceAsyncTests.CreateAsync_EmptyId_AssignsNewId
✔ CrudServiceAsyncTests.CreateAsync_FromManyThreadsInParallel_StoresAllElements
✔ CrudServiceAsyncTests.CreateAsync_NewElement_ReturnsTrueAndStoresElement
✔ CrudServiceAsyncTests.CreateAsync_Null_ThrowsArgumentNullException
✔ CrudServiceAsyncTests.Enumeration_IteratesElementsInOrder
✔ CrudServiceAsyncTests.Enumeration_ModifyingServiceDuringForeach_DoesNotThrow
✔ CrudServiceAsyncTests.GetTotalPages_ReturnsRoundedUpPageCount(10,3,4)
✔ CrudServiceAsyncTests.GetTotalPages_ReturnsRoundedUpPageCount(10,5,2)
✔ CrudServiceAsyncTests.GetTotalPages_ReturnsRoundedUpPageCount(0,5,0)
✔ CrudServiceAsyncTests.LoadAsync_MissingFile_ReturnsFalse
✔ CrudServiceAsyncTests.ReadAllAsync_EmptyService_ReturnsEmptyCollection
✔ CrudServiceAsyncTests.ReadAllAsync_InvalidPaging_ThrowsArgumentOutOfRangeException(0,5)
✔ CrudServiceAsyncTests.ReadAllAsync_InvalidPaging_ThrowsArgumentOutOfRangeException(-1,5)
✔ CrudServiceAsyncTests.ReadAllAsync_InvalidPaging_ThrowsArgumentOutOfRangeException(1,0)
✔ CrudServiceAsyncTests.ReadAllAsync_InvalidPaging_ThrowsArgumentOutOfRangeException(1,-3)
✔ CrudServiceAsyncTests.ReadAllAsync_Paging_ReturnsExpectedPage(1,3,[0, 1, 2])
✔ CrudServiceAsyncTests.ReadAllAsync_Paging_ReturnsExpectedPage(2,3,[3, 4, 5])
✔ CrudServiceAsyncTests.ReadAllAsync_Paging_ReturnsExpectedPage(4,3,[9])
✔ CrudServiceAsyncTests.ReadAllAsync_Paging_ReturnsExpectedPage(5,3,[])
✔ CrudServiceAsyncTests.ReadAllAsync_Paging_ReturnsExpectedPage(1,20,[0, 1, 2, 3, 4, ...])
✔ CrudServiceAsyncTests.ReadAllAsync_ReturnsAllElementsInInsertionOrder
✔ CrudServiceAsyncTests.ReadAsync_MissingId_ThrowsKeyNotFoundException
✔ CrudServiceAsyncTests.RemoveAsync_ExistingElement_ReturnsTrueAndDeletesIt
✔ CrudServiceAsyncTests.RemoveAsync_MissingElement_ReturnsFalse
✔ CrudServiceAsyncTests.SaveAsync_CalledConcurrently_AllSucceedAndFileIsValid
✔ CrudServiceAsyncTests.SaveAsync_ThenLoadAsync_PreservesDerivedEmployeeTypes
✔ CrudServiceAsyncTests.SaveAsync_ThenLoadAsync_RestoresElementsInSameOrder
✔ CrudServiceAsyncTests.UpdateAsync_ConcurrentUpdatesOfSameElement_AllSucceed
✔ CrudServiceAsyncTests.UpdateAsync_ExistingElement_ReplacesItAndKeepsPosition
✔ CrudServiceAsyncTests.UpdateAsync_MissingElement_ReturnsFalse

Test Count: 39, Passed: 39, Failed: 0, Warnings: 0, Inconclusive: 0, Skipped: 0
```

4. Відповіді на контрольні запитання

1. Що таке інтерфейс ICrudServiceAsync<T>? Яке його призначення? Чому доцільно використовувати generic тип T для CRUD-сервісу?

`ICrudServiceAsync<T>` — це узагальнений інтерфейс (контракт) асинхронного сервісу доступу до даних. Він визначає операції CRUD (Create, Read, Update, Delete), а також читання з пагінацією і збереження у файл. Усі методи повертають `Task` / `Task<T>`, тобто виконуються асинхронно. Інтерфейс наслідує `IEnumerable<T>`, тому сервіс можна перебирати через `foreach` і використовувати з LINQ.

Призначення: відокремити контракт від реалізації. Код консольного застосунку працює з будь-якою реалізацією (у пам'яті, з файлом, з базою даних), реалізацію легко замінити або підмінити заглушкою в тестах. Асинхронні методи не блокують потік, що їх викликав, на час довгих операцій, зокрема запису у файл.

Generic тип T дає змогу написати одну реалізацію для всіх сутностей: той самий `CrudServiceAsync<T>` працює з `Employee`, `GameProject` і `BugReport`. Код не дублюється, а перевірка типів відбувається під час компіляції: не потрібні приведення типів чи `object`, і в сервіс працівників неможливо додати проєкт. Обмеження `where T : class, IEntity` гарантує, що в кожного елемента є `Id`.

```
var employees = new CrudServiceAsync<Employee>("employees.json");
var projects  = new CrudServiceAsync<GameProject>("projects.json");
await employees.CreateAsync(Programmer.CreateNew());   // ок
// await employees.CreateAsync(GameProject.CreateNew()); — помилка компіляції
```

2. Що таке процес та потік, яка між ними різниця? Що таке багатопотокове виконання?

Процес — екземпляр програми, що виконується; об'єкт операційної системи, контейнер ресурсів: власний віртуальний адресний простір (код, дані, купа), відкриті файли та дескриптори, ідентифікатор PID і щонайменше один потік. Процеси ізольовані один від одного.

Потік (thread) — найменша одиниця виконання, яку планує ОС. Потік існує всередині процесу, має власний стек, регістри та лічильник команд, але спільну з іншими потоками процесу пам'ять і ресурси.

	Процес	**Потік**
Пам'ять	власний ізольований адресний простір	спільна пам'ять процесу, власний лише стек
Створення і перемикання	дорогі	значно дешевші
Взаємодія	через механізми IPC (канали, сокети, файли)	напряму через спільні змінні (потрібна синхронізація)
Наслідки збою	не зачіпає інші процеси	необроблений виняток завершує весь процес

Багатопотокове виконання — одночасна робота кількох потоків одного процесу. На багатоядерному процесорі потоки виконуються справді паралельно, на одному ядрі ОС перемикає їх за квантами часу (витискальна багатозадачність). У лабораторній `Parallel.For` розподіляє створення 1000 працівників між кількома потоками пулу потоків.

3. Що таке асинхроність, яка різниця між асинхронністю та багатопотоковістю?

Асинхронність — це спосіб виконання, за якого програма запускає операцію і не чекає її завершення, блокуючи потік. Поки триває операція (читання файлу, мережевий запит, таймер), потік повертається до іншої роботи, а продовження методу виконується, коли результат готовий.

Асинхронність	**Багатопотоковість**
про *очікування без блокування*	про *одночасне виконання* коду кількома потоками
найкраще для операцій вводу-виводу (I/O-bound): файли, мережа, БД	найкраще для обчислень (CPU-bound): розрахунки, генерація даних
може працювати навіть з одним потоком; під час асинхронного I/O потік узагалі не зайнятий	завжди використовує кілька потоків і ядер процесора
у лабораторній: `SaveAsync` / `LoadAsync`, `SemaphoreSlim.WaitAsync`, `Task.Delay`	у лабораторній: `Parallel.For`, `Parallel.ForEach`, окремі `Thread`

Ці підходи поєднуються: `Parallel.ForEachAsync` виконує асинхронне тіло циклу паралельно, а `Task.Run` переносить обчислення в потік пулу, щоб його можна було очікувати через `await`.

4. Для чого використовуються ключові слова async/await?

`async` позначає метод як асинхронний: компілятор перетворює його на скінченний автомат (`IAsyncStateMachine`) і дозволяє використовувати всередині `await`. Такий метод повертає `Task`, `Task<T>`, `ValueTask<T>` (або `void` — тільки для обробників подій).

`await` асинхронно очікує завершення задачі. Якщо задача ще не завершена, метод повертає керування тому, хто його викликав, а решта методу виконується як продовження, коли задача завершиться. `await` також «розгортає» результат (`Task<T>` → `T`) і перекидає виняток, тому помилки обробляються звичайним `try/catch`.

Завдяки async/await асинхронний код виглядає як послідовний — без колбеків і ручного `ContinueWith`. Самі по собі ці слова нових потоків не створюють. У бібліотечному коді (`CrudServiceAsync`) використано `ConfigureAwait(false)`, щоб продовження не поверталось у вихідний контекст синхронізації.

```
public async Task<bool> SaveAsync()
{
    // чекаємо семафор, не блокуючи потік
    await _fileLock.WaitAsync().ConfigureAwait(false);
    try
    {
        await using var stream = new FileStream(..., useAsync: true);
        await JsonSerializer.SerializeAsync(stream, GetSnapshot(), JsonOptions);
        ...
        return true;
    }
    finally { _fileLock.Release(); }
}
```

5. Чим відрізняється Task від ValueTask?

Task / Task<T>	**ValueTask / ValueTask<T>**
клас (reference-тип), кожна задача — об'єкт у купі	структура (value-тип); якщо результат готовий одразу, виділення пам'яті немає

Task / Task<T>	ValueTask / ValueTask<T>
можна очікувати кілька разів, зберігати, передавати в `Task.WhenAll` / `WhenAny`	можна очікувати лише один раз; не можна очікувати одночасно з кількох місць
`.Result` безпечний після завершення	`.Result` не можна читати до завершення; для складних сценаріїв — `.AsTask()`
універсальний вибір за замовчуванням	оптимізація для «гарячих» методів, які часто завершуються синхронно (кеш, буфер)

У `CrudServiceAsync` операції в пам'яті завершуються одразу (`Task.FromResult`), тож `ValueTask<bool>` міг би зекономити виділення пам'яті. Проте інтерфейс із завдання визначає `Task`, а результати `CreateAsync` у програмі збираються в `Task.WhenAll`, для чого потрібен саме `Task`.

6. Що таке thread-safe колекція? Які приклади таких колекцій у .NET ви знаєте?

Thread-safe (потокобезпечна) колекція — колекція, методи якої можна викликати одночасно з різних потоків без зовнішнього блокування. Її внутрішній стан при цьому не псується, а операції на кшталт «додати, якщо немає» виконуються атомарно. Звичайні `List<T>` і `Dictionary<TKey, TValue>` такими не є: паралельний `List.Add` губить елементи або кидає виняток.

Приклади з простору імен `System.Collections.Concurrent`:

- `ConcurrentDictionary<TKey, TValue>` — словник з атомарними `TryAdd`, `TryUpdate`, `TryRemove`, `GetOrAdd`, `AddOrUpdate` (сховище `CrudServiceAsync`, словник семафорів для файлів);
- `ConcurrentQueue<T>` — черга FIFO (журнал рендер-ферми в демонстрації `Semaphore`);
- `ConcurrentStack<T>` — стек LIFO;
- `ConcurrentBag<T>` — невпорядкований набір (задачі створення багів у `Parallel.ForEach`);
- `BlockingCollection<T>` — обгортка для схеми «виробник — споживач» з блокуванням і обмеженням місткості.

Також потокобезпечні незмінні колекції (`System.Collections.Immutable`: `ImmutableList<T>`, `ImmutableDictionary<TKey, TValue>`) і канали `System.Threading.Channels`.

7. Для чого використовуються примітиви синхронізації, такі як lock, Semaphore, AutoResetEvent?

Примітиви синхронізації координують роботу потоків: забезпечують **взаємне виключення** (до спільного ресурсу одночасно звертається лише дозволена кількість потоків) і **сигналізацію** (один потік чекає, поки інший повідомить про подію). Без них виникають стан гонки та пошкодження даних.

Примітив	Призначення
`lock` (`Monitor`)	критична секція для одного потоку в межах процесу; `Monitor.TryEnter` — з тайм-аутом
`Mutex`	те саме, що lock, але об'єкт ядра ОС; іменований м'ютекс працює між процесами

Примітив	Призначення
`Semaphore` / `SemaphoreSlim`	пропускає одночасно не більше N потоків; `SemaphoreSlim` легший і має `WaitAsync` для async-коду
`AutoResetEvent`	сигнал, що пропускає рівно один потік, який чекає, і автоматично скидається
`ManualResetEvent(Slim)`	«ворота»: після `Set()` пропускає всіх, поки не викликано `Reset()`
`CountdownEvent`, `Barrier`	очікування N сигналів; синхронізація потоків по фазах
`ReaderWriterLockSlim`	багато читачів одночасно або один письменник
`Interlocked`	атомарні операції над числами без блокування

Усі ці примітиви продемонстровано в розділі 7 результатів виконання програми.

8. Як забезпечити безпеку при одночасному зверненні кількох потоків до спільного ресурсу?

- **Уникати спільного змінюваного стану:** локальні змінні, незмінні (immutable) об'єкти, копії даних для кожного потоку.
- **Потокобезпечні колекції** замість звичайних (`ConcurrentDictionary` у сервісі).
- **Атомарні операції** `Interlocked` для лічильників (лічильник `Employee`, порядкові номери в сервісі).
- **Критичні секції** `lock` / `Monitor` навколо коду, що змінює спільні дані: блокувати приватний `readonly` об'єкт (не `this` і не рядки), тримати блокування якомога коротше, захоплювати кілька блокувань завжди в однаковому порядку, щоб уникнути взаємного блокування (deadlock).
- **SemaphoreSlim з WaitAsync** в асинхронному коді: всередині `lock` не можна використовувати `await` (захист файлу в `SaveAsync`).
- **ReaderWriterLockSlim** для даних, які часто читають і рідко змінюють.
- **Робота зі знімком:** перебирати копію колекції, а не саму колекцію, щоб паралельні зміни не заважали (`GetEnumerator` сервісу).

9. Як за допомогою LINQ можна отримати мінімальне, максимальне та середнє значення певної властивості?

Методи розширення LINQ `Min`, `Max` і `Average` приймають лямбда-вираз (селектор), який вибирає потрібну властивість:

```
decimal minSalary = employees.Min(e => e.BaseSalary);
decimal maxSalary = employees.Max(e => e.BaseSalary);
decimal avgSalary = employees.Average(e => e.BaseSalary);
// для int результат Average — double
double avgCommits = programmers.Average(p => p.CommitsCount);
// MaxBy повертає сам елемент з максимумом (.NET 6+)
Programmer topCoder = programmers.MaxBy(p => p.CommitsCount)!;
```

Для порожньої послідовності `Min`, `Max` і `Average` кидають `InvalidOperationException`, тому перед ними варто перевірити `Any()`. Також є `Sum`, `Count`, `MinBy` / `MaxBy`, а для кількох показників за один прохід — `Aggregate`. Саме так у програмі побудовано таблиці статистики (розділ 2 результатів).

10. У чому різниця між методами Select, Where, Aggregate, OrderBy?

Метод	Що робить	Приклад
`Select`	проєкція: перетворює кожен елемент на інше значення (1 → 1)	`employees.Select(e => e.FullName)`
`Where`	фільтрація: залишає елементи, що задовольняють умову	`bugs.Where(b => !b.IsFixed)`
`OrderBy`	сортування за ключем за зростанням (`OrderByDescending`, `ThenBy`)	`projects.OrderByDescending(p => p.Budget)`
`Aggregate`	згортка: накопичує всі елементи в одне значення	`list.Aggregate(0m, (sum, e) => sum + e.BaseSalary)`

`Select`, `Where` та `OrderBy` виконуються відкладено (deferred execution): запит обчислюється лише під час перебору й повертає нову послідовність. `Aggregate` виконується одразу й повертає одне значення.

11. Які переваги використання LINQ у порівнянні з класичними циклами?

- **Декларативність і читабельність:** описується *що* потрібно отримати, а не *як*; один рядок `employees.Average(e => e.BaseSalary)` замінює цикл з тимчасовими змінними.
- **Менше коду і помилок:** немає індексів, лічильників і проміжних списків.
- **Композиція:** методи об'єднуються в ланцюжки (`GroupBy → OrderBy → Select → Take`).
- **Єдиний синтаксис** для різних джерел: колекції, бази даних (EF Core перетворює LINQ на SQL), XML.
- **Відкладене виконання** і проста паралелізація через PLINQ (`AsParallel()`).

Недоліки: невеликі накладні витрати (делегати, ітератори), ризик повторного перебору одного запиту, складніше налагодження. У критичних за швидкодією місцях класичний цикл може бути швидшим.

12. Що станеться, якщо два потоки одночасно спробують зберегти колекцію у файл?

Без синхронізації можливі такі наслідки:

- другий потік отримає `IOException` («файл використовується іншим процесом»), якщо перший відкрив файл без спільного доступу;
- якщо спільний запис дозволено, дані двох потоків перемішаються або файл обріжеться, і JSON стане пошкодженим;
- якщо колекцію змінюють під час серіалізації, звичайна колекція кине `InvalidOperationException` («Collection was modified»), а у файл може потрапити неузгоджений стан.

У `CrudServiceAsync` це вирішено так. `SaveAsync` спершу чекає `SemaphoreSlim`, один для кожного шляху до файлу й спільний для всіх екземплярів сервісу, тому збереження виконуються по черзі. Серіалізується знімок колекції, а запис іде в тимчасовий файл, який потім атомарно замінює основний: навіть при збої старий файл лишається цілим. У програмі 10 одночасних `SaveAsync` завершились успішно (10 з 10), а модульний тест перевіряє 20 одночасних збережень і коректність файлу після них. Для захисту між різними процесами можна додатково використати іменований `Mutex`.

13. Як працює Parallel.For і у яких випадках його краще використовувати?

`Parallel.For(from, to, body)` розбиває діапазон індексів на частини і виконує тіло циклу в кількох потоках пулу одночасно (потік, що викликав, теж бере участь). Метод блокує виклик, доки не завершаться всі ітерації. Порядок ітерацій не гарантується. Через `ParallelOptions` можна задати `MaxDegreeOfParallelism` і `CancellationToken`, через `ParallelLoopState` — достроково зупинити цикл (`Break` / `Stop`). Винятки з ітерацій збираються в `AggregateException`.

Коли використовувати: для незалежних ітерацій з обчислювальною роботою (CPU-bound) достатнього обсягу: генерація й обробка даних, розрахунки, обробка зображень. Спільні дані в тілі циклу мають бути потокобезпечними: у лабораторній це `ConcurrentDictionary` сервісу, масив задач з окремим індексом для кожної ітерації та `Interlocked`.

Коли не варто: для асинхронного вводу-виводу (краще `Parallel.ForEachAsync` або `Task.WhenAll`), для дуже коротких ітерацій (накладні витрати більші за виграш) і якщо ітерації залежать одна від одної або важливий порядок.

14. Що таке пагінація і як вона реалізована у вашій системі?

Пагінація — поділ великого набору даних на сторінки фіксованого розміру, щоб повертати клієнту лише потрібну частину (номер сторінки + розмір сторінки). Це зменшує обсяг пам'яті та переданих даних і пришвидшує відображення списків.

У системі пагінацію реалізує метод `ReadAllAsync(int page, int amount)`:

- сторінки нумеруються з 1; `page < 1` або `amount < 1` дає `ArgumentOutOfRangeException`;
- елементи впорядковуються за порядковим номером додавання, тому сторінки стабільні навіть для `ConcurrentDictionary`, який сам порядку не зберігає;
- повертається `Skip((page - 1) * amount).Take(amount)`; сторінка за межами даних — порожня колекція;
- `GetTotalPages(amount)` повертає кількість сторінок (округлення вгору).

```
long skip = (long)(page - 1) * amount;
List<T> pageItems = skip >= int.MaxValue
    ? new List<T>()
    : GetSnapshot().Skip((int)skip).Take(amount).ToList();
```

Приклад з програми: 1000 працівників при розмірі сторінки 5 дають 200 сторінок, сторінка 201 порожня.